

TECHNICAL IMPLEMENTATION REPORT

Service Request Intake & SLA Tracking System

Automation Engineer (n8n) - Practical Technical Assessment

Figure placeholder - Optional candidate photo

Prepared by	Sara
Submission date	29 September 2026
Technology stack	Appsmith, n8n, PostgreSQL, Docker Compose

Contents

Contents	2
1. Executive summary	3
2. What was implemented	3
Employee request intake.....	3
Automatic processing and classification	3
Duplicate prevention and safe retries.....	4
Request creation and identifiers	4
3. SLA implementation	4
4. Automation-team queue.....	5
5. n8n workflow design	5
6. Escalation and reporting	6
7. Data integrity and reliability.....	6
8. Security considerations	7
9. Testing and verification	7
10. Running the solution	8
11. Trade-offs and next steps.....	9
12. Closing note.....	9

1. Executive summary

Figure placeholder - Solution architecture

I built a working service-request intake and SLA-tracking solution to replace the department's spreadsheet-based process. The solution uses Appsmith for the user interface, n8n for orchestration, PostgreSQL for transactional business logic and persistence, and Mailpit for safe local email testing.

The implementation covers the complete path from request submission to operational follow-up:

Employee -> Appsmith -> n8n -> PostgreSQL -> SLA tracking -> Team queue

It also adds escalation, reporting, audit history, API-level idempotency support, and automated tests. My main design goal was to make the system predictable under failure: validation, duplicate detection, request-ID generation, SLA calculation, storage, and audit logging happen atomically, so a request is either committed completely or not written at all.

The delivered stack was tested locally. Both SQL test suites passed, and the end-to-end smoke test passed every success and error path exercised by the project.

2. What was implemented

Employee request intake

The employee-facing Appsmith form captures the requester name and email, department, request type, title, description, declared priority, and optional contextual information such as company and needed-by date. P1 requests require a justification.

The UI performs basic validation for required values and email format. The server repeats validation because client-side checks alone cannot be trusted. The submission is sent only to an n8n webhook; Appsmith does not hold database credentials.

Automatic processing and classification

The n8n submission workflow receives the request, applies deterministic keyword-based classification, calls a single PostgreSQL function, routes the result by outcome, returns an appropriate HTTP response, and sends a confirmation email after the response.

The classifier uses the requester's selected request type as its starting point. It changes the category only when the title and description clearly support one alternative. Both the declared request type and the resulting category are stored, together with the reason for classification. This makes the decision explainable and auditable.

I chose rules instead of an LLM for this version because they are deterministic, inexpensive, fast, and keep request content inside the local environment. The n8n Code node can later be replaced with an LLM-based classifier without changing the database or API contract.

Duplicate prevention and safe retries

The solution distinguishes between two situations that can look similar to a user:

1. Technical retry: If the same idempotency_key is received again, the API returns the original request as a replay instead of creating another row.
2. Business duplicate: If the same requester email and normalized title already belong to an active request, the new request is rejected and the existing request ID is returned.

Title normalization lowercases text, removes punctuation, collapses repeated whitespace, and normalizes Persian/Arabic forms of **ی** and **ک**. Duplicate protection is enforced by both an advisory transaction lock and a partial unique index. This means two simultaneous submissions cannot bypass the rule.

A resolved or cancelled request does not block a future request with the same title, because the same business need may legitimately occur again.

Request creation and identifiers

Each request receives a human-readable ID in the required format:

REQ-YYYY-NNNNNN

A per-year counter is incremented inside the same transaction as the request insert. This prevents collisions and avoids consuming an ID when a transaction fails.

The core request record stores requester details, the original and classified request types, declared and effective priority, timestamps, SLA deadline, status, assignment, escalation level, and optional contextual fields. A separate append-only event table records creation, assignment, status changes, escalation, priority changes, and blocked duplicates.

3. SLA implementation

The SLA is calculated when the request is created and stored in due_at as a timezone-aware timestamp.

Priority			Ta
P1	4 clock hours	Exactly four hours after creation, including weekends	
P2	1 business day	17:00 on the next business day	
P3	3 business days	17:00 on the third business day	
P4	5 business days	17:00 on the fifth business day	

Saturday and Sunday are skipped for P2-P4. Public holidays are intentionally outside the assessment scope. Business timezone and end-of-day time are configuration values rather than hard-coded

constants. All stored timestamps use PostgreSQL timestampz; the local timezone is applied only when calculating or displaying the 17:00 deadline.

Open requests receive one of the required indicators:

- ON TRACK: more than 24 hours remain.
- AT RISK: the deadline is within the next 24 hours.
- BREACHED: the deadline has passed.

For completed or resolved requests, the final SLA result is calculated against resolved_at, not the current time. This preserves the historical result as MET or BREACHED. Cancelled work is reported separately.

4. Automation-team queue

The queue API returns requests in operational order, prioritizing the most urgent open work. Status, department, and SLA filters are combined with logical AND, so team members can narrow the list precisely. Reference values include all departments and statuses specified in the assessment.

The team can assign a request, change its status, and add a note. These changes are performed through an audited PostgreSQL function. Invalid statuses return a clear validation response, and unknown request IDs return a not-found response.

The employee and team interfaces are separated. This reduces accidental exposure of the full request queue and supports different access policies in a production Appsmith workspace.

5. n8n workflow design

Six workflows divide the automation into focused responsibilities:

Workflow	
Service Request - Submit	Receive, classify, store, respond, and confirm by email
Service Request - Queue	Return the filtered live queue
Service Request - Update	Assign requests and change status with audit history
SLA - Escalation	Check SLA state every 15 minutes and notify the correct audience
SLA - Reporting	Serve on-demand reports and email a scheduled daily report

Workflow		
Ops - Error alerts	Notify the team lead when another workflow fails	

Webhook workflows return explicit status codes: 201 for creation, 200 for a replay or successful read/update, 409 for a business duplicate, 404 for an unknown request, 422 for invalid input, and 500 for an unexpected error.

Database operations retry where it is safe to do so. On a server error, the user-facing response is sent first; the workflow is then deliberately failed so the centralized error workflow records and reports the failure. Confirmation email is also sent after the successful HTTP response, so an email outage cannot turn a valid stored request into a failed submission from the user's perspective.

6. Escalation and reporting

Although the detailed core requirements stop at the filtered queue, I implemented the escalation and reporting stages shown in the requested high-level journey.

The escalation workflow runs every 15 minutes:

- Level 1 warns the Automation team when a request becomes AT RISK.
- Level 2 alerts the team lead when a request becomes BREACHED.

The escalation level is stored on the request and can only move upward. Therefore, repeated schedule executions do not repeatedly email the same alert. FOR UPDATE SKIP LOCKED prevents overlapping runs from blocking each other.

The report includes totals and breakdowns by department, priority, category, and status. SLA compliance is calculated as met requests divided by requests whose result is already decided. Open requests that are not yet due are excluded from that ratio because their final result is not yet known; cancelled requests are reported but excluded from compliance.

7. Data integrity and reliability

Business-critical write operations are encapsulated in PostgreSQL functions. A submission performs the following steps within one database transaction:

3. Validate and normalize the input.
4. Serialize competing submissions for the same logical request.
5. Check the idempotency key.
6. Check for an active business duplicate.
7. Generate the request ID.
8. Calculate the SLA deadline.
9. Insert the request and its audit event.

An unexpected exception rolls back the complete operation. Database constraints provide a final safety layer for email normalization, field length, priorities, request-ID format, foreign keys, duplicate requests, and deadline validity.

This placement of rules is an intentional trade-off. The n8n canvas is slightly less self-contained, but the guarantees are stronger, the operations are testable, and retry behavior is safer. n8n still exposes each business outcome as a visible branch.

8. Security considerations

The local solution follows several useful security boundaries:

- Appsmith communicates with n8n and has no direct database credentials.
- n8n passes payloads to PostgreSQL as a parameterized JSON value rather than constructing SQL from request text.
- The database accepts only whitelisted fields and validates reference values again.
- Secrets are supplied through environment variables and are not embedded in workflow exports.
- Mailpit captures all test email locally, so development notifications do not leave the machine.
- Employee and team-facing interfaces are separate.

The current webhooks are unauthenticated, which is acceptable only for this isolated local assessment environment. In production I would add webhook authentication, restrict network exposure, enable TLS, apply role-based access, rotate secrets, and share the team application only with the appropriate Appsmith group.

9. Testing and verification

The database test suite covers:

- P1 clock-hour calculation.
- P2-P4 business-day calculation across weekdays and weekends.
- The Tehran/UTC date boundary and the 17:00 local deadline.
- SLA status calculation.
- Title normalization.
- Successful creation, idempotent replay, duplicate rejection, invalid input, and resubmission after closure.
- Combined queue filters and urgency ordering.
- Assignment and status-change auditing.
- One-time escalation and escalation from level 1 to level 2.
- SLA report aggregation.

The end-to-end smoke test then exercises the running n8n webhooks, including both happy and unhappy paths.

Verified result on 28 September 2026:

Test area	
SQL operations tests	Passed
SQL SLA and duplicate tests	Passed
Create request (201)	Passed
Idempotent replay (200)	Passed
Duplicate rejection (409)	Passed
Invalid submission (422)	Passed
P1 without justification (422)	Passed
Combined queue filters (200)	Passed
Unknown request (404)	Passed
Unknown status (422)	Passed
Update/cancel request (200)	Passed
SLA report (200)	Passed

The smoke-test request was cancelled automatically after verification to keep the queue clean.

10. Running the solution

The complete local environment is defined with Docker Compose:

- Appsmith: <http://localhost:8080>
- n8n: <http://localhost:5678>
- Mailpit: <http://localhost:8025>
- PostgreSQL: localhost:5432

After creating `.env` from `.env.example`, the stack can be started with `docker compose up -d`. On first initialization, PostgreSQL creates the schema, reference data, functions, and demo records. The setup script imports and publishes the n8n workflows and creates the PostgreSQL and local SMTP credentials. The smoke-test script runs the SQL tests and calls the public workflow endpoints.

The repository also includes both Appsmith application exports:

- `appsmith/Service Desk.json` - employee request-submission interface.

- appsmith/Service Desk - Automation Team.json - team queue, request management, and SLA reporting interface.

They can be imported directly from the Appsmith workspace using Create new -> Import. Both exports include the REST datasource definition that points to the n8n service inside the Docker network.

11. Trade-offs and next steps

The implementation is deliberately practical rather than over-engineered. The main remaining production improvements are:

- Connect the form to the API's idempotency feature. The backend already accepts an idempotency_key and safely replays the original response when that key is reused. The current exported Appsmith form does not generate or send this key, so a double-click is still protected by the business duplicate rule but is shown as a duplicate rather than as an already-received request. The intended improvement is to generate one UUID per form session and reuse it for retries of the same submission.
- Add authentication and authorization to every webhook.
- Add server-side pagination for a large queue.
- Replace Mailpit with an approved SMTP service.
- Add a holiday calendar if business SLAs later require it.
- Add monitoring and delivery retry/outbox handling for stronger email guarantees.
- Add integration tests for the Appsmith pages themselves.
- Upgrade PostgreSQL according to the supported version guidance of the deployed n8n release.

The escalation design currently favors at-most-once notification: the escalation level is committed before email delivery. This avoids repeated messages, but a final email failure requires the operations alert to be acted upon. For a higher-criticality production system, I would use a transactional outbox so state changes and notification delivery can be retried independently with at-least-once guarantees.

12. Closing note

I approached this assessment as a small operational system rather than a simple form connected to a table. The implementation therefore focuses on traceability, safe retries, concurrency, explicit error outcomes, and testable business rules, while keeping the user journey straightforward.

The result is a working foundation that meets the requested intake, duplicate prevention, persistence, SLA, and filtering requirements and extends naturally into assignment, escalation, reporting, and operational support.